CONVENÇÃO VETOR-LINHA

Ponto = linha: **(x, y, 1)** em 2D e **(x, y, z, 1)** em 3D. **P' = P · M**. Translação na **última linha**; escala na diagonal. θ > 0 = **anti-horário**, visto do semieixo positivo olhando a origem (mão direita).

Vetor-coluna = tudo transposto: P' = M · P, translação na última coluna, ordem lida da direita para a esquerda. Muita sentença falsa é a versão vetor-coluna de uma fórmula certa.

MATRIZES 2D (3×3)

$$R(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad T(t_x, t_y) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{bloco 2×2 de } R: \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

MATRIZES 3D (BLOCO 3×3 DA 4×4)

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & \sin\theta \\ 0 & -\sin\theta & \cos\theta \end{bmatrix} \quad R_y = \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$$

Na 4×4: bloco linear no canto superior esquerdo, 4ª linha = (t_x, t_y, t_z, 1), 4ª coluna = (0, 0, 0, 1). S(s_x, s_y, s_z) na diagonal. **90°:** R: (x, y) → (−y, x) · R_x: (x, y, z) → (x, −z, y) · R_y: → (z, y, −x) · R_z: → (−y, x, z). **180°** em 2D: (−x, −y).

TRIGONOMETRIA

θ	30°	45°	60°	90°	120°	135°	150°	180°
cos	.866	.707	.5	0	−.5	−.707	−.866	−1
sin	.5	.707	.866	1	.866	.707	.5	0

tan 30° = 0.577 · tan 45° = 1 · tan 60° = 1.732 · √2 = 1.414 · √3 = 1.732 · √5 = 2.236 · √3/2 = 0.866

3-4-5: arctan ¾ = 36.87° (cos 0.8, sin 0.6) · arctan 4/3 = 53.13° (cos 0.6, sin 0.8) · arctan 2 = 63.43° (cos 1/√5 = 0.447, sin 2/√5 = 0.894).

cos(−θ) = cos θ, sin(−θ) = −sin θ. Com m = tan α: **cos 2α = (1 − m²)/(1 + m²)**, **sin 2α = 2m/(1 + m²)**.

VETORES

$a \cdot b = a_x b_x + a_y b_y + a_z b_z$ · $\|a\| = \sqrt{(a \cdot a)}$ · unitário = $a/\|a\|$ · $a \perp b \iff a \cdot b = 0$.

$$a \times b = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x)$$

a × b é perpendicular a a e a b; a × b = −(b × a); vetores paralelos ⇒ a × b = 0.

SISTEMAS DE COORDENADAS

Tela: origem no canto superior esquerdo, +y para baixo.
2D: 8 orientações de eixos, todas equivalentes por rotação.
3D: não; há mão esquerda e mão direita. Com +x à direita e +y para cima: **esquerda → +z entra na tela**; **direita → +z sai da tela**. Polegar +x, indicador +y, médio +z.

Trocar de mão exige **reflexão** (ex.: S(1, 1, −1), det −1). Rotação (det +1) nunca troca.

Cadeia: Objeto → Model → Mundo → View → Câmera → Projection → Recorte (clip) → ÷w → NDC [−1, 1] → Viewport → Tela. O espaço UV (textura) é **paralelo**, fora da cadeia.

A câmera não se move: move-se o mundo para o referencial dela. Frustum = pirâmide vista pela câmera; clipping = objeto parcialmente na tela; oclusão = escondido atrás de outro.

Objeto · upright · mundo

Upright = **origem do objeto + eixos do mundo**. Objeto ↔ upright: só rotação. Upright ↔ mundo: só translação.

Modelo: **gira primeiro, translada depois** (rotação na origem é linear; em torno de outro ponto é afim). Para a câmera: translada primeiro, gira depois.

$M = R \cdot T(t)$ → bloco = R, última linha = t
$P_{mundo} = P_{obj} \cdot M$ $P_{upright} = P_{obj} \cdot R = P_{mundo} - t$
$M^{-1} = T(-t) \cdot R^T$ → última linha = $-t \cdot R^T$, com $R^T = R(-\theta)$
$P_{obj} = (Q_{mundo} - t) \cdot R^T$

Polares, cilíndricas, esféricas

Polares: x = r cos θ, y = r sin θ; r = √(x² + y²), θ = arctan(y/x) ajustando o quadrante. Uso: texturas radiais, gradientes.

Cilíndricas (eixo z): x = r cos θ, y = r sin θ, z = z. Uso: simetria axial (canos, garrafas, colunas). **Esféricas:** orbit camera, luzes, skybox, planetas.

Orbit: eye = C + ρ · (cos φ sin θ, sin φ, cos φ cos θ)
θ = azimute de +z para +x; φ = elevação sobre o plano xz
Cilíndricas, eixo vertical por C: r = ρ cos φ, y = ρ sin φ, θ = arctan(Δx/Δz); confere com Δx = r sin θ, Δz = r cos θ
f = (C − eye)/ρ

Variar só ρ muda a distância e mantém a direção. **φ = 90°:** f paralelo a up ⇒ f × up = 0 ⇒ r indefinido (limitar φ a 89° ou trocar up).

Editor cartesiano → tela

Origem no centro (c_x, c_y) = (L/2, H/2), +y para cima, 1 unidade = s pixels.

$$M = S(s, -s) \cdot T(c_x, c_y) = \begin{bmatrix} s & 0 & 0 \\ 0 & -s & 0 \\ c_x & c_y & 1 \end{bmatrix}$$

$$M^{-1} = \begin{bmatrix} 1/s & 0 & 0 \\ 0 & -1/s & 0 \\ -c_x/s & c_y/s & 1 \end{bmatrix}$$

ida: $x_t = s \cdot x + c_x$, $y_t = -s \cdot y + c_y$
volta: $x = (x_t - c_x)/s$, $y = -(y_t - c_y)/s$

det do bloco = $-s^2$: módulo = fator de área; sinal negativo = reflexão (anti-horário vira horário). Ponto em polares: converter para (x, y) antes.

TRANSFORMAÇÕES

Lineares (cabem em 2×2): escala, rotação, reflexão, cisalhamento. **Afim:** translação (é soma), só vira matriz com coordenada homogênea.

w = 0 = direção (ponto no infinito): sofre o bloco linear, **não** a translação.

Afim 4×4 = bloco linear **L** (3×3 superior esquerdo) + translação **t** (4ª linha).

Escala: $s_x = s_y$ uniforme; diferente distorce; > 1 amplia; entre 0 e 1 reduz; < 0 reflete. S(−1, 1) = reflexão no eixo y; S(1, −1) = no eixo x.

Determinante de L

Rotação +1 · escala $s_x \cdot s_y$ · reflexão −1 · cisalhamento 1. **|det| = fator de área** (volume em 3D); **det < 0 inverte a orientação** (sentido dos vértices). det(A·B) = det A · det B. Translação não muda área.

Duas reflexões em retas pela origem = **rotação** pelo dobro do ângulo entre as retas (det +1), não outra reflexão.

Área ABC = ½ · $|(b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x)|$
Área' = |det L| · Área

Composição e ordem

P · R · T = gira e depois translada (em vetor-coluna: T · R · P).

$$\begin{bmatrix} L_1 & 0 \\ t_1 & 1 \end{bmatrix} \cdot \begin{bmatrix} L_2 & 0 \\ t_2 & 1 \end{bmatrix} = \begin{bmatrix} L_1 \cdot L_2 & 0 \\ t_1 \cdot L_2 + t_2 & 1 \end{bmatrix}$$

Comuta: rotações 2D na origem, R(α)·R(β) = R(α + β); translações entre si; escala uniforme com rotação.

Não comuta: translação com rotação ou com escala (mesmo uniforme); rotações 3D em eixos distintos; escala não uniforme com rotação; cisalhamentos.

P·S(2,2)·T(1,0) = (2x + 1, 2y), mas P·T(1,0)·S(2,2) = (2x + 2, 2y). Mesmas matrizes em outra ordem: mesmo det e mesma área, posição diferente.

Pivô K (rotação/escala fora da origem)

$M = T(-K) \cdot L \cdot T(K)$ ⇒ bloco = L, **t = K − K·L**

$L = R(\theta)$, S ou R(θ)·S(s, s) = $s \cdot \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$. K é ponto fixo ($K \cdot M = K$). Toda distância ao pivô é multiplicada por s.

3D, eixo paralelo a x por (0, a, b): T(0, −a, −b) · $R_x(\theta)$ · T(0, a, b); a distância do ponto ao eixo se conserva. Eixo arbitrário: translada à origem, alinha a um eixo coordenado, aplica $R_x/R_y/R_z$, desfaz na ordem inversa.

Reflexão na reta y = m·x + b

α = arctan m
M = T(0, −b) · R(−α) · S(1, −1) · R(α) · T(0, b)

$$L = \begin{bmatrix} \cos 2\alpha & \sin 2\alpha \\ \sin 2\alpha & -\cos 2\alpha \end{bmatrix} \quad t = (0, -b) \cdot L + (0, b)$$

Conferir: ponto médio de PP' está na reta; PP' · (1, m) = 0; det = −1; M·M = I (M^{-1} = M).

Cisalhamento (vetor-linha)

$x' = x + k \cdot y$: $\begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$ $y' = y + k \cdot x$: $\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$

det = 1: retângulo vira paralelogramo de mesma área. Inversa: troca k por −k. $H_1 \cdot H_2 \neq H_2 \cdot H_1$.

Inversas

$T(t)^{-1} = T(-t)$ · $R(\theta)^{-1} = R(-\theta) = R^T$ (det +1) · $S^{-1} = S(1/s_x, 1/s_y, 1/s_z)$, fatores ≠ 0 · reflexão^{-1} = ela mesma · $(A \cdot B)^{-1} = B^{-1} \cdot A^{-1}$.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = 1/(ad - bc) \cdot \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad \begin{bmatrix} L & 0 \\ t & 1 \end{bmatrix}^{-1} = \begin{bmatrix} L^{-1} & 0 \\ -t \cdot L^{-1} & 1 \end{bmatrix}$$

Decompor L = R · S

1) L = bloco 3×3, t = 4ª linha. 2) Norma de cada linha = s; normas iguais e linhas ortogonais ⇒ L = s·R (escala uniforme). 3) R = L/s; em R_z, cos θ = R_{11} e sin θ = R_{12}. 4) $L^{-1} = R^T/s$. 5) P = (P' − t) · L^{-1}; conferir P·M = P'.

Orientação

Orientação ≠ direção: o vetor não fixa o giro (twist); são 3 graus de liberdade. **Heading** = eixo vertical (guinada); **pitch** = eixo lateral (arfagem); **bank** = eixo longitudinal (rolamento). Ângulos de Euler sofrem **gimbal lock**; quatérnios evitam.

EXEMPLOS RESOLVIDOS (DA LISTA)

Orbit (Q8): C (2, 1, −3), ρ = 10, θ = φ = 36.87° ⇒ deslocamento (4.8, 6, 6.4), eye (6.8, 7, 3.4); cilíndricas (8, 36.87°, 6); f = (−0.48, −0.6, −0.64).

Objeto → mundo (Q9): M = $R_y(120°) \cdot T(18, 0, 10)$. P_{obj} (2, 5, 1) → mundo (17.87, 5, 7.77); upright = mundo − t = (−0.13, 5, −2.23). Q_{mundo} (20, 3, 7): Q − t = (2, 3, −3), ·$R_y(-120°)$ = (1.60, 3, 3.23).

Editor → tela (Q10): 1280×720, s = 40 ⇒ M = S(40, −40)·T(640, 360). (3, −2.5) → (760, 460); pixel (1000, 100) → (9, 6.5); polar (5, 143.13°) = (−4, 3) → (480, 240); det = −1600.

Ordem (Q15): S(2,1)·R(90°)·T(3,−2): (x, y) → (−y + 3, 2x − 2). T(3,−2)·R(90°)·S(2,1): (x, y) → (−2y + 4, x + 3). det = 2 nos dois casos: área 3 → 6.

Pivô (Q16): K = (3, 3), L = R(30°)·S(2,2) = $\begin{bmatrix} \sqrt{3} & 1 \\ -1 & \sqrt{3} \end{bmatrix}$, t = K − K·L = (6 − 3√3, −3√3) ≈ (0.80, −5.20). B (4, 2) → (5.73, 2.27).

Reflexão (Q17): y = 2x + 1 ⇒ cos 2α = −0.6, sin 2α = 0.8; L = $\begin{bmatrix} -0.6 & 0.8 \\ 0.8 & 0.6 \end{bmatrix}$, t = (−0.8, 0.4); (3, 2) → (−1, 4).

Eixo deslocado (Q18): T(0,−2,−3)·$R_x(90°)$·T(0,2,3): bloco $R_x(90°)$, t = (0, 5, 1); (4, 5, 3) → (4, 2, 6). Q (1, 2, 3): Q·R_x·R_y = (2, −3, −1) ≠ Q·R_y·R_x = (3, 1, 2).

Cisalhamentos (Q19): $H_1 \cdot H_2$ = $\begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0.25 \\ 0 & 1 \end{bmatrix}$ = $\begin{bmatrix} 1 & 0.25 \\ 0.5 & 1.125 \end{bmatrix}$; inversa = $\begin{bmatrix} 1.125 & -0.25 \\ -0.5 & 1 \end{bmatrix}$; área 4 se conserva.

Decomposição (Q20): linhas (1.2, 1.6, 0), (−1.6, 1.2, 0), (0, 0, 2): norma 2 ⇒ s = 2, cos θ = 0.6, sin θ = 0.8, θ = 53.13°; t = (5, −3, 4). M^{-1}: bloco $R^T/2$, última linha (−0.3, 2.9, −2); (7, 1, 10) → (2.2, 0.4, 3). Direção (1, 1, 0, 0) → (−0.4, 2.8, 0, 0).

Slide 04 (fixação): (3, 2)·T(−2, 5) = (1, 7) · (4, −1)·S(2, 3) = (8, −3) · (2, 0)·R(60°) = (1, 1.73) · (1, 1)·S(2,2)·R(180°) = (−2, −2) · (5, 5) girado 90° em torno de (3, 3) = (1, 5). Triângulo A (1,1), B (3,1), C (2,3) com S(1.5,1.5)·R(45°)·T(2,1): A' (2, 3.12), B' (4.12, 5.24), C' (0.94, 6.30).

Roteiro de cálculo: 1) escrever as matrizes na ordem de aplicação; 2) compor bloco L e translação t; 3) aplicar aos pontos; 4) conferir (ponto fixo, det e área, $M \cdot M^{-1} = I$); 5) 2 casas decimais quando não for exato.

PEGADINHAS DE V/F

Certo é: recorte antes da rasterização · Phong é local · bump não muda a silhueta · pintor vai do longe ao perto · aliasing é amostrar abaixo de 2f · rotação não troca a mão · tela tem origem em cima à esquerda · UV fica fora da cadeia · translação na última linha · pivô = T(−c)·R·T(c) · translação não comuta com escala · 2 reflexões = rotação · um vetor não define orientação · olhando −z, V = S(1, 1, −1) · V^{-1} não é a transposta inteira · P não faz a perspectiva sem ÷w' · dobrar n não muda $x_{ndc} \cdot z_{ndc}$ não é linear · dois extremos fora não garantem rejeição.

Slide 04 tem trechos em vetor-coluna (cisalhamento, forma geral 4×4, ordem do pivô no exercício 5): na lista, use as transpostas.

CÂMERA (MATRIZ V)

$f = (target - eye) / \|target - eye\|$
$r = (f \times up) / \|f \times up\| \quad u = r \times f$
$up = (0, 1, 0) \Rightarrow f \times up = (-f_z, 0, f_x)$

f = frente, r = direita, u = vertical real; unitários e perpendiculares entre si. Ordem dos produtos: **f × up** e **r × f**. up paralelo a f ⇒ base indefinida.

$d = P_{mundo} - eye$
$x_c = d \cdot r,\ y_c = d \cdot u,\ z_c = d \cdot f$ ($z_c > 0$ à frente da câmera)

$$V = \begin{bmatrix} r_x & u_x & f_x & 0 \\ r_y & u_y & f_y & 0 \\ r_z & u_z & f_z & 0 \\ -eye \cdot r & -eye \cdot u & -eye \cdot f & 1 \end{bmatrix}$$

$$V^{-1} = \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ f_x & f_y & f_z & 0 \\ eye_x & eye_y & eye_z & 1 \end{bmatrix}$$

V = T(−eye) · R, com r, u, f nas **colunas**. $V^{-1} = R^T \cdot T(eye)$: r, u, f nas **linhas** e última linha = eye. **Não** é a transposta da 4×4 inteira.

Câmera na origem olhando para −z: r = (1, 0, 0), u = (0, 1, 0), f = (0, 0, −1) ⇒ **V = S(1, 1, −1)**, não a identidade ($z_c = -z$). Câmera alinhada aos eixos: bastaria transladar.

Conferir sempre das duas formas: produtos escalares e $P_{mundo} \cdot V$.

PROJEÇÃO (MATRIZ P)

Ortográfica: raios paralelos; preserva paralelismo e medidas (CAD, plantas técnicas). **Perspectiva:** raios convergem no centro de projeção; o distante fica menor (jogos, realismo).

Ponto de fuga: paralelas que não são paralelas ao plano de projeção convergem; as paralelas ao plano continuam paralelas.

$x_p = n \cdot x_c / z_c \quad y_p = n \cdot y_c / z_c$ (semelhança de triângulos)

Dividir por z_c **não é linear**: P copia z_c para w' e a **divisão homogênea** vem depois (hardware). É a única etapa não linear; o recorte 3D acontece antes dela.

$aspect = L/H \quad top = n \cdot \tan(FOV_y/2) \quad right = top \cdot aspect$

$$P = \begin{bmatrix} n/right & 0 & 0 & 0 \\ 0 & n/top & 0 & 0 \\ 0 & 0 & A & 1 \\ 0 & 0 & B & 0 \end{bmatrix} \quad A = (f + n)/(f - n)$$

$B = -2 \cdot f \cdot n / (f - n)$
$P_{clip} = (x_c \cdot n/right,\ y_c \cdot n/top,\ A \cdot z_c + B,\ z_c)$
$P_{ndc} = (x', y', z') / w'$, com $w' = z_c$

$x_{ndc} = x_p/right = x_c / (z_c \cdot \tan(FOV_y/2) \cdot aspect)$
$y_{ndc} = y_p/top = y_c / (z_c \cdot \tan(FOV_y/2))$
$z_{ndc} = A + B/z_c$

Aqui f = distância do plano far. O "1" (3ª linha, 4ª coluna) copia z_c para w'. top e right = meia-altura e meia-largura do plano near.

n se cancela em x_{ndc} e y_{ndc}: mudar n só afeta z_{ndc} e o recorte. $z_c = n \rightarrow -1$; $z_c = f \rightarrow +1$. z_{ndc} depende de $1/z_c$ (hiperbólica): a precisão do depth buffer se concentra perto do near; n muito pequeno piora o z-fighting ao longe.
NDC = cubo $[-1, 1]^3$; desacopla recorte e viewport da resolução. Mudar a janela altera W, não V; P só muda se o aspect mudar.

VIEWPORT (MATRIZ W)

Aula (janela inteira, z passa direto):
$x_t = (x_{ndc} + 1)/2 \cdot L \quad y_t = (1 - y_{ndc})/2 \cdot H$

$$W = \begin{bmatrix} L/2 & 0 & 0 & 0 \\ 0 & -H/2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ L/2 & H/2 & 0 & 1 \end{bmatrix}$$

Geral: canto superior esquerdo (x_0, y_0), tamanho L×H, z de [−1, 1] para [0, 1]:
$x_t = x_0 + (x_{ndc} + 1)/2 \cdot L$
$y_t = y_0 + (1 - y_{ndc})/2 \cdot H \quad z_j = (z_{ndc} + 1)/2$

$$W = \begin{bmatrix} L/2 & 0 & 0 & 0 \\ 0 & -H/2 & 0 & 0 \\ 0 & 0 & ½ & 0 \\ x_0 + L/2 & y_0 + H/2 & ½ & 1 \end{bmatrix}$$

$$W^{-1} = \begin{bmatrix} 2/L & 0 & 0 & 0 \\ 0 & -2/H & 0 & 0 \\ 0 & 0 & 2 & 0 \\ -(2x_0 + L)/L & (2y_0 + H)/H & -1 & 1 \end{bmatrix}$$

volta: $x_{ndc} = (x_t - x_0 - L/2)/(L/2)$
$y_{ndc} = -(y_t - y_0 - H/2)/(H/2) \quad z_{ndc} = 2 \cdot z_j - 1$

O termo −H/2 inverte y. Cantos: NDC (−1, 1) → canto superior esquerdo; (1, −1) → inferior direito; z = −1 → 0 (near), z = +1 → 1 (far).

Matrizes W diferem por 3 convenções: y invertido ou não; origem em (0, 0) ou em (x_{min}, y_{min}); z direto ou mapeado para uma faixa.

MATRIZ ÚNICA M = V · P · W

1) V·P (linha de V × coluna de P). 2) (V·P)·W. 3) $P_{mundo} \cdot M = (x', y', z', w')$. 4) Dividir x', y', z' por w', que vale z_c.

W antes da divisão é legítimo: W é afim, com última coluna $(0, 0, 0, 1)^T$, e não altera w' ⇒ $(P_{clip} \cdot W)/w' = (P_{clip}/w') \cdot W$. M é pré-multiplicada uma vez por quadro.

O target está no eixo de visada ($x_c = y_c = 0$) ⇒ projeta no centro da janela (L/2, H/2).

Exemplo da aula: eye (4, 0, 3), target (0, 0, 0), up (0, 1, 0), n = 1, f = 100, FOV_y = 90°, 800×600 ⇒ f = (−0.8, 0, −0.6), r = (0.6, 0, −0.8), u = (0, 1, 0); top = 1, right = 4/3. P (0, 2, −5) → câmera (4, 2, 8) → near (0.5, 0.25) → clip (3, 2, 6.14, 8) → NDC (0.375, 0.25, 0.77) → código 0000 → tela (550, 225).

Lista (Q25–28): eye (2, 7, 9), target (2, 1, 1) ⇒ f = (0, −0.6, −0.8), r = (1, 0, 0), u = (0, 0.8, −0.6); última linha de V = (−2, −0.2, 11.4). FOV_y = 90°, n = 2, f = 22, 1024×768 ⇒ top = 2, right = 8/3, n/right = 0.75, n/top = 1, A = 1.2, B = −4.4. P (6, 3, −0.5) → câmera (4, 2.5, 10) → clip (3, 2.5, 7.6, 10) → NDC (0.3, 0.25, 0.76) → tela (665.6, 288). Q_1 (−12, 1, 8) → clip (−9, 1, 5.2, 8): x < −w, plano esquerdo. Q_2 (0.5, 0.5, 1) → clip (0.375, 0.5, −3.2, 1): z < −w, plano near.

Viewport (Q27): 800×600 em (100, 50), z → [0, 1] ⇒ $x_t = 400 \cdot x + 500$, $y_t = -300 \cdot y + 350$, $z_j = 0.5 \cdot z + 0.5$. NDC (0.3, 0.25, 0.76) → (620, 275, 0.88); pixel (300, 500) com z = 0.5 → NDC (−0.5, −0.5, 0).

RECORTE

Cohen-Sutherland (segmentos)

bit 0 (0001) x < x_{min} · bit 1 (0010) x > x_{max} · bit 2 (0100) y < y_{min} · bit 3 (1000) y > y_{max}

$y > y_{max}$	1001	1000	1010
dentro em y	0001	**0000**	0010
$y < y_{min}$	0101	0100	0110

OR = 0 → aceitação trivial. **AND ≠ 0** → rejeição trivial (fora do mesmo lado). Caso contrário: interseção com uma borda, novo código, repetir o teste.

Dois extremos fora não bastam: 0001 AND 0010 = 0000 (lados opostos; o segmento pode cruzar a janela).

borda x = x_b: $y = y_1 + (y_2 - y_1)/(x_2 - x_1) \cdot (x_b - x_1)$
borda y = y_b: $x = x_1 + (x_2 - x_1)/(y_2 - y_1) \cdot (y_b - y_1)$

Ordem das bordas na lista: esquerda, direita, abaixo, acima. Um extremo com 2 bits pode exigir 2 interseções. Mostrar OR, AND e cada novo código.

Sutherland-Hodgman (polígonos)

Uma borda por vez; a saída de uma é a entrada da próxima. Percorre as arestas s → p, incluindo último → primeiro.

dentro → dentro	emite p	**dentro → fora**	emite a interseção
fora → dentro	interseção e p	**fora → fora**	não emite nada

Dentro: $x \geq x_{min}$, $x \leq x_{max}$, $y \geq y_{min}$, $y \leq y_{max}$. Só as bordas violadas por algum vértice mudam a lista. Um triângulo pode virar pentágono.

$t = (x_b - x_s)/(x_p - x_s) \quad I = S + t \cdot (P - S)$
Shoelace: $A = ½ \cdot |\Sigma (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i)|$, vértices em ordem, fechando no primeiro

C-S (Q29): janela [10, 50]×[10, 40]; A (0, 20) = 0001, B (60, 50) = 1010: OR 1011, AND 0000 ⇒ recortar. Reta y = 20 + 0.5x: A → (10, 25); B → (50, 45), código 1000 → (40, 40). Resultado: (10, 25)–(40, 40).

S-H (Q30): triângulo (2, 2), (14, 4), (4, 12) na janela $[0, 10]^2$. Borda x = 10: I_1 (10, 3.33), I_2 (10, 7.2). Borda y = 10: J_1 (6.5, 10), J_2 (3.6, 10). Pentágono I_2, J_1, J_2, V_1, I_1; área 47.37 (original 58).

Espaço homogêneo (3D)

Antes da divisão por w: **−w ≤ x, y, z ≤ w** (evita erros com pontos atrás da câmera). Plano violado: x < −w esquerdo · x > w direito · y < −w inferior · y > w superior · z < −w near ($z_c < n$) · z > w far ($z_c > f$).

FUNDAMENTOS (TEORIA)

Pipeline conceitual: 1 Modelagem → 2 Transformações → 3 Recorte → 4 Rasterização → 5 Shading → 6 Exibição. As etapas 1–3 preparam a geometria; 4–6 produzem a imagem.
Malhas: triângulos; só aproximam curvas. **Paramétricas** (Bézier, B-splines, NURBS): funções contínuas com pontos de controle; seguem em uso (CAD). **Voxels:** grade 3D com um valor por célula (científica, médica). **CSG:** primitivas + operações booleanas; não é malha (converter ou traçar raios).
Cor: RGB aditivo (telas) · CMYK subtrativo (impressão) · HSV/HSL = matiz, saturação, valor (edição intuitiva) · YUV (vídeo). **Gamma:** a percepção não é linear; a correção faz a exibição corresponder à intensidade pretendida.
Iluminação local: Phong = ambiente + difusa + especular; Blinn-Phong é a variação eficiente. **Global** (luz indireta): ray tracing, path tracing, radiosity. Bump/normal mapping: só perturbam normais (silhueta igual); displacement: desloca a geometria.
Visibilidade: Z-buffer guarda a profundidade mais próxima por pixel; independe da ordem (opacos); padrão em GPU. Back-face culling descarta faces de costas (≈ metade em objeto fechado); não resolve oclusão entre objetos. Pintor: do mais distante ao mais próximo; falha em ciclos e interpenetração. BSP: árvore binária do espaço.
Rasterização: rápida, tempo real, base de OpenGL/DirectX/Vulkan. **Ray/path tracing:** fisicamente mais preciso, historicamente offline, tempo real com RTX.
Aliasing: amostragem abaixo de 2× a maior frequência (Nyquist). Mipmapping: texturas reduzidas pré-calculadas para objetos distantes (menos aliasing, mais desempenho). GPU massivamente paralela; shaders vertex, fragment, compute.

INTRODUÇÃO (TEORIA)

Whirlwind (MIT, início dos anos 1950; pedido da Marinha por um simulador de voo): 1º a usar light pen para selecionar aeronaves no CRT; sucessor SAGE. **Ivan Sutherland:** pai da CG; TX-2 com CRT de 9"; 1º sistema gráfico interativo.
Spacewar! 1961, Steve Russell (MIT), ~200 homens-hora, PDP-1 (1º SO multiusuário simultâneo). Higinbotham 1958 (Brookhaven). Bushnell = Atari. Death Race 1976 (1ª controvérsia). Space Invaders 1978 (grava pontuações). Asteroids 1979 (iniciais). **Pac-Man** 1980: Namco, Toru Iwatani, 18 meses, para eliminar o tema dos "tiros". Donkey Kong: Nintendo, Miyamoto. Adventure (Atari 2600): 1º easter egg, Warren Robinett. Tetris 1985: Pajitnov.
Visible Human: proposto em 1986 pela Biblioteca Nacional de Medicina (EUA); masculino 1994, feminino 1995; referência anatômica e teste de algoritmos de imagens médicas.
Áreas: CG = objeto → imagem · Visão = imagem → objeto (reconhecimento facial) · Processamento de Imagens = imagem → imagem · Modelagem = objeto → objeto. RV/RA exigem tempo real. Robótica: simular sensores e rotas antes do robô físico.
CG = geração, manipulação, representação e exibição de imagens por computador; fica entre matemática, física, arte e engenharia de software. Base matemática: álgebra linear, transformações, geometria projetiva, cálculo e trigonometria (curvas, interpolação), quatérnios.
Aplicações: visão computacional (reconhecimento facial, detecção de objetos, reconstrução 3D) · RV/RA · jogos e simulação · imagens médicas (tomografia e ressonância; diagnóstico, planejamento cirúrgico) · visualização científica (meteorologia, fluidos) · modelagem e animação 3D · robótica · APIs gráficas (OpenGL, Vulkan) · IA generativa de conteúdo visual · renderização distribuída em GPUs · impressão 3D.